

Day 48 — Line-by-Line Syntax Explanation

day48_transfer_learning.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 7 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula, a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system.

MATH box legend: $\times$ = matrix multiply / elementwise times W, b = weights/bias m = batch size l_r = learning rate $d(\text{name})$ = gradient of loss w.r.t. that tensor

Segment 0 — Imports and Shared Helpers (reused verbatim from Days 39-47)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)

def relu(z): return np.maximum(0, z)
def drelu(z): return (z > 0).astype(z.dtype)
def tanh(z): return np.tanh(z)
def dtanh(z): return 1 - np.tanh(z) ** 2
def softmax(z): ...
def one_hot(y, n_classes): ...
def cross_entropy_loss(probs, y_onehot): ...
def he_scale(n_in): return np.sqrt(2.0 / n_in)
def conv2d_forward_mc(X, W, b): ...
def conv2d_backward_mc(dZ, X, W): ...
def maxpool_forward(A, size=2, stride=2): ...
def maxpool_backward(dOut, A, idx_cache, size=2, stride=2): ...
def make_junction_dataset_rgb(...): ...
class TwoBlockConvNetRGB: ...
```

None of this is new today -- every function and class here is a character-for-character copy of Day 43-47's own code, unchanged. TwoBlockConvNetRGB (2,024 params at default sizes) plays THREE roles today: the source network that gets pretrained, the 'from scratch' control, and the architecture every transfer strategy warm-starts a fresh copy of. make_junction_dataset_rgb is called TWICE today with DIFFERENT geometry/noise arguments -- once for the source domain, once for the target -- but the function body itself never changes.

Segment 1 — Transfer Learning Machinery (NEW today)

MATH

freeze_conv=True: update only the head

$W1 \leftarrow lr \cdot dW1, b1 \leftarrow lr \cdot db1, W2 \leftarrow lr \cdot dW2, b2 \leftarrow lr \cdot db2$

freeze_conv=False: ALSO update both conv blocks

$Wc1 \leftarrow lr \cdot dWc1, bc1 \leftarrow lr \cdot dbc1, Wc2 \leftarrow lr \cdot dWc2, bc2 \leftarrow lr \cdot dbc2$

```
def backward_transfer(net, y_onehot, freeze_conv):
    m = y_onehot.shape[0]
    dZ2 = net.probs - y_onehot
    dW2 = net.A1.T @ dZ2 / m
    db2 = dZ2.sum(axis=0) / m
    dA1 = dZ2 @ net.W2.T
    dZ1 = dA1 * dtanh(net.Z1)
    dW1 = net.flat.T @ dZ1 / m
    db1 = dZ1.sum(axis=0) / m

    updates = [(net.W2, dW2), (net.b2, db2),
               (net.W1, dW1), (net.b1, db1)]
```

The head's own gradients (dW2, db2, dW1, db1) are computed the SAME way regardless of freeze_conv -- this is exactly the first half of TwoBlockConvNetRGB's own backward() method, copied unchanged. The `updates` list starts with just these four (parameter, gradient) pairs; whether anything else gets appended to it is what freeze_conv controls, decided next.

```
if not freeze_conv:
    dflat = dZ1 @ net.W1.T
    dpooled2 = dflat.reshape(net.pooled2.shape)
    dAc2 = maxpool_backward(dpooled2, net.Ac2, net.masks2, 2, 2)
    dZc2 = dAc2 * drelu(net.Zc2)
    dWc2, dbc2, dpooled1 = conv2d_backward_mc(
        dZc2, net.pooled1, net.Wc2)
    dAc1 = maxpool_backward(dpooled1, net.Ac1, net.masks1, 2, 2)
    dZc1 = dAc1 * drelu(net.Zc1)
    dWc1, dbc1, _ = conv2d_backward_mc(dZc1, net.X, net.Wc1)
    updates += [(net.Wc1, dWc1), (net.bc1, dbc1),
               (net.Wc2, dWc2), (net.bc2, dbc2)]

for p, g in updates:
    p -= net.lr * g
```

This ENTIRE block -- every maxpool_backward and conv2d_backward_mc call -- is inside `if not freeze\_conv:`. When freeze_conv=True, none of this code runs at all: not just the final update, the GRADIENT COMPUTATION itself is skipped. This is only correct because the conv layers are the very first layers in the network -- there's no earlier layer that still needs `dpooled1` or `dAc1` to compute ITS OWN gradient. The final loop only ever touches whatever ended up in `updates` -- 4 pairs when frozen, 8 when not.

```
def train_transfer(net, X, y, epochs, freeze_conv, n_classes=4):
    y_oh = one_hot(y, n_classes)
    losses = []
    for _ in range(epochs):
        probs = net.forward(X)
        losses.append(cross_entropy_loss(probs, y_oh))
        backward_transfer(net, y_oh, freeze_conv)
    return losses
```

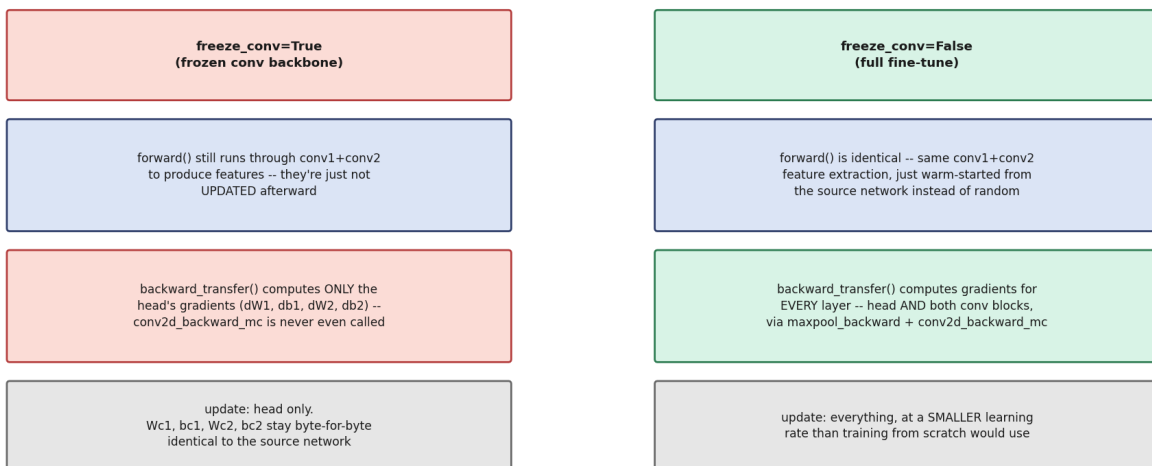
The same per-epoch loop shape as every prior day's train() method -- the only difference is calling the new backward_transfer(..., freeze_conv) instead of net.backward(...). Note that net.forward(X) is called EVERY epoch regardless of freeze_conv -- freezing only ever affects the BACKWARD pass; the frozen conv layers still do real work producing features on every single forward call.

```
def copy_conv_weights(source_net, target_net):
    target_net.Wc1 = source_net.Wc1.copy()
    target_net.bc1 = source_net.bc1.copy()
    target_net.Wc2 = source_net.Wc2.copy()
    target_net.bc2 = source_net.bc2.copy()

def copy_all_weights(source_net, target_net):
    copy_conv_weights(source_net, target_net)
    target_net.W1 = source_net.W1.copy()
    target_net.b1 = source_net.b1.copy()
    target_net.W2 = source_net.W2.copy()
    target_net.b2 = source_net.b2.copy()
```

Both functions use .copy() -- without it, target_net.Wc1 and source_net.Wc1 would be the SAME array in memory, and training the target network would silently also mutate the source network. copy_all_weights() is written to reuse copy_conv_weights() rather than repeating those four lines, keeping the 'copy just the conv layers' and 'copy everything' cases from ever drifting out of sync.

SYSTEM DIAGRAM



SYSTEM DIAGRAM: the same backward_transfer() call, freeze_conv=True vs. False.

Segment 2 — Gradient Check (both freeze_conv modes, plus a literal freeze check)

```
def demo_gradient_check():
    rng = np.random.RandomState(0)
    for freeze_conv in [False, True]:
        net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=16,
                                  hidden=5, n_classes=3, lr=0.01, seed=1)
        X = rng.randn(4, 3, 16, 16) * 0.5
        y_oh = one_hot(np.array([0, 1, 2, 0]), 3)
        net.forward(X)
        ... # analytic dW1, dW2 always computed;
        ... # analytic dWc1, dWc2 ALSO computed when not freeze_conv
```

The check runs TWICE, once per freeze_conv value, in the same loop -- img_size=16 (smaller than the real 30) keeps it fast while still exercising both conv blocks for real. Which analytic gradients get computed mirrors backward_transfer() itself: dW1/dW2 always, dWc1/dWc2 only when freeze_conv is False -- there's no point checking a gradient that backward_transfer never even computes.

```
params_to_check = ["W1", "W2"]
if not freeze_conv:
    params_to_check = ["W1", "W2", "Wc1", "Wc2"]
eps = 1e-4
for name in params_to_check:
    param = getattr(net, name)
    for _ in range(3):
        idx = tuple(rng.randint(0, s) for s in param.shape)
        orig = param[idx]
        param[idx] = orig + eps
        lp = cross_entropy_loss(net.forward(X), y_oh)
        param[idx] = orig - eps
        lm = cross_entropy_loss(net.forward(X), y_oh)
        param[idx] = orig
        numeric = (lp - lm) / (2 * eps)
```

getattr(net, name) looks up net.W1, net.W2, net.Wc1, or net.Wc2 by its STRING name -- this is what lets the same loop body check a different, shorter list of tensors depending on freeze_conv, without writing the nudge-and-measure logic out twice. Each checked weight is nudged by +eps then -eps and the full forward() pipeline is rerun both times, exactly the same full-pipeline convention every prior day's gradient check has used since Day 44's isolated-check bug.

```
net = TwoBlockConvNetRGB(seed=5)
Wc1_before, Wc2_before = net.Wc1.copy(), net.Wc2.copy()
X, y = make_junction_dataset_rgb(n_per_class=5, seed=1)
train_transfer(net, X, y, epochs=20, freeze_conv=True)
print(np.array_equal(Wc1_before, net.Wc1))
print(np.array_equal(Wc2_before, net.Wc2))
```

A second, DIFFERENT kind of check than a gradient check: this verifies `freeze_conv=True` doesn't just skip a gradient in theory -- `np.array_equal` confirms the actual `Wc1/Wc2` NumPy arrays are byte-for-byte identical before and after 20 real training epochs. Both come back True in this lesson's real run.

Segment 3 — Pretraining the Source Network

```
def pretrain_source():
    X_train, y_train = make_junction_dataset_rgb(
        n_per_class=50, size=30, arm=8, noise=0.4,
        channel_noise=0.15, center_jitter=2, seed=42)
    X_test, y_test = make_junction_dataset_rgb(
        n_per_class=20, size=30, arm=8, noise=0.4,
        channel_noise=0.15, center_jitter=2, seed=999)
    source_net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30,
        hidden=8, lr=0.08, seed=42)
    source_net.train(X_train, y_train, epochs=400)
    return source_net
```

Nothing new here syntactically -- this is this series' exact standard training recipe since Day 43, arm=8 and all, calling TwoBlockConvNetRGB's own verbatim `.train()` method directly in one clean call. What's new is what this trained network gets used FOR next: it becomes the ``source_net`` argument every transfer strategy below receives.

Segment 4 — The Transfer Experiment (5 strategies, same 20 images)

```
def demo_transfer_experiments(source_net):
    X_tgt_train, y_tgt_train = make_junction_dataset_rgb(
        n_per_class=5, size=30, arm=11, noise=0.4,
        channel_noise=0.3, center_jitter=3, seed=123)
    X_tgt_test, y_tgt_test = make_junction_dataset_rgb(
        n_per_class=20, size=30, arm=11, noise=0.4,
        channel_noise=0.3, center_jitter=3, seed=777)
    results = {}

    results["zero-shot"] = dict(
        train_acc=source_net.accuracy(X_tgt_train, y_tgt_train),
        test_acc=source_net.accuracy(X_tgt_test, y_tgt_test),
    )
```

The target domain call to `make_junction_dataset_rgb` uses DIFFERENT `arm/channel_noise/center_jitter` values than the source domain call in `pretrain_source()` -- same function, different arguments, a real visual shift. Zero-shot needs no training loop at all: `source_net` is just evaluated directly on target data it has never seen, via the same `.accuracy()` method every network in this series already has.

```
scratch_net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30,
    hidden=8, lr=0.08, seed=7)
losses_scratch = scratch_net.train(
    X_tgt_train, y_tgt_train, epochs=400)
results["from-scratch"] = dict(
    train_acc=scratch_net.accuracy(X_tgt_train, y_tgt_train),
    test_acc=scratch_net.accuracy(X_tgt_test, y_tgt_test),
)
```

`seed=7` here (not `seed=42`) is deliberate -- it's used for EVERY fresh head created in this function (scratch, frozen+fresh head, and the starting point before `copy_all_weights` overwrites it), so any differences between strategies come from the STRATEGY, not from different random head initializations. `scratch_net` ignores `source_net` completely -- it's the control condition.

```
frozen_fresh = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30,
    hidden=8, lr=0.08, seed=7)
copy_conv_weights(source_net, frozen_fresh)
train_transfer(frozen_fresh, X_tgt_train, y_tgt_train,
    epochs=400, freeze_conv=True)
```

`copy_conv_weights` overwrites `frozen_fresh`'s randomly-initialized conv layers with the source network's trained ones -- but its head (`W1, b1, W2, b2`) is left exactly as `TwoBlockConvNetRGB.__init__` created it: fresh and random. `freeze_conv=True` in `train_transfer` means those overwritten conv weights will stay untouched for all 400 epochs; only the fresh head learns anything.


```

frozen_warm = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30,
    hidden=8, lr=0.08, seed=7)
copy_all_weights(source_net, frozen_warm)
losses_frozen_warm = train_transfer(frozen_warm,
    X_tgt_train, y_tgt_train, epochs=400, freeze_conv=True)

```

The ONLY difference from the previous block: `copy_all_weights` instead of `copy_conv_weights`. This one extra word is the entire code difference between 'frozen+fresh head' and 'frozen+warm head' -- the head starts from the source network's already-trained head instead of a random one, then continues training from there (`freeze_conv` is still `True`, so only the head moves).

```

finetune_lrs = [0.08, 0.03, 0.01, 0.005]
finetune_results = {}
losses_finetune_best = None
for lr in finetune_lrs:
    ft_net = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30,
        hidden=8, lr=lr, seed=7)
    copy_all_weights(source_net, ft_net)
    losses_ft = train_transfer(ft_net, X_tgt_train, y_tgt_train,
        epochs=400, freeze_conv=False)
    finetune_results[lr] = dict(
        train_acc=ft_net.accuracy(X_tgt_train, y_tgt_train),
        test_acc=ft_net.accuracy(X_tgt_test, y_tgt_test))
    if lr == 0.01:
        losses_finetune_best = losses_ft
best_lr = max(finetune_lrs,
    key=lambda lr: finetune_results[lr]["test_acc"])

```

`freeze_conv=False` here -- every layer, conv and head, keeps training. The loop tries FOUR different learning rates on otherwise-identical warm-started networks, and `max(..., key=lambda lr: ...)` picks out whichever one actually reached the best real `test_acc`, rather than assuming in advance which learning rate would work best. ``lr == 0.01`` happens to be this run's winner, so that's the one whose loss curve gets saved for the comparison plot.

SYSTEM DIAGRAM

Same 20 target images, 5 strategies: what's kept vs. thrown away vs. trained

strategy	conv layers	head	target test_acc
1. zero-shot	kept from source (no training at all)	kept from source (no training at all)	0.6375
2. from-scratch	random init, TRAINED	random init, TRAINED	0.3625 (worst)
3. frozen + fresh head	frozen from source	random init, TRAINED	0.4500
4. frozen + warm head	frozen from source	kept from source, TRAINED	0.6625 (best, tied)
5. fine-tune (lr=0.01)	kept from source, TRAINED	kept from source, TRAINED	0.6625 (best, tied)

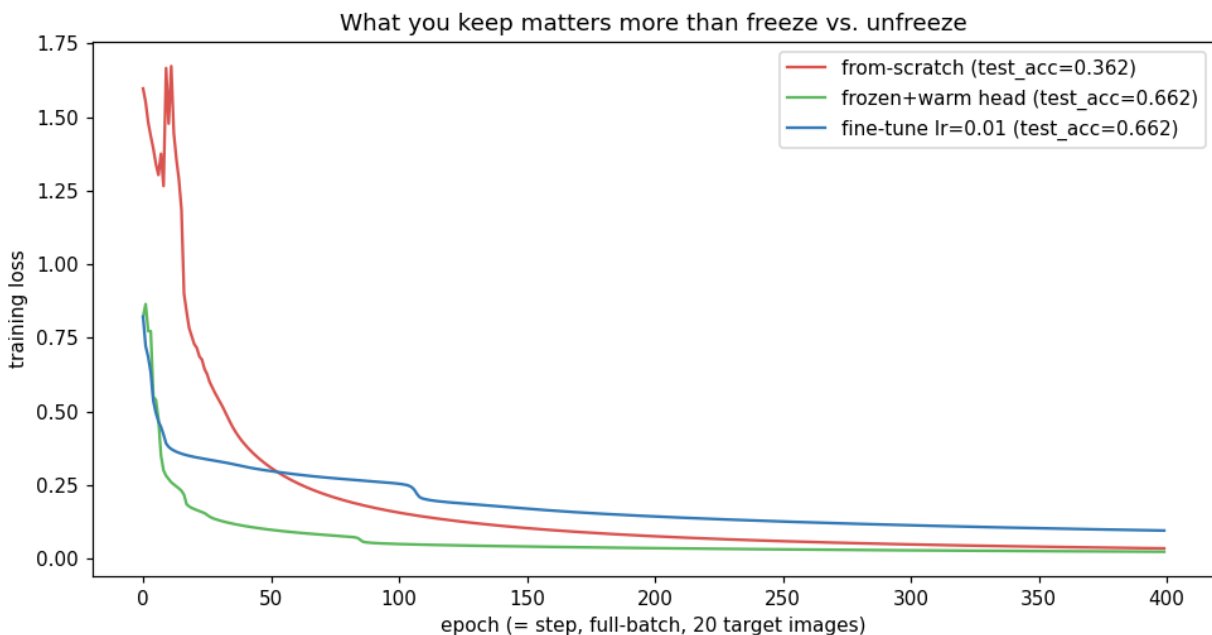
SYSTEM DIAGRAM: what each of the 5 code paths above keeps, discards, and trains.

Segment 5 — The Comparison Plot

```
def make_comparison_plot(losses_scratch, losses_frozen_warm,
                        losses_finetune, test_acc_scratch, test_acc_frozen,
                        test_acc_finetune):
    plt.figure(figsize=(9, 4.8))
    plt.plot(losses_scratch,
             label=f"from-scratch (test_acc={test_acc_scratch:.3f})",
             color="#d9534f")
    plt.plot(losses_frozen_warm,
             label=f"frozen+warm head (test_acc={test_acc_frozen:.3f})",
             color="#5cb85c")
    plt.plot(losses_finetune,
             label=f"fine-tune lr=0.01 (test_acc={test_acc_finetune:.3f})",
             color="#337ab7")
```

Only 3 of the 5 strategies are plotted (zero-shot and frozen+fresh head are omitted) -- zero-shot never trains at all, so it has no loss curve to plot, and frozen+fresh head was left out simply to keep the chart to three readable lines. The three chosen span the full range of outcomes: the worst strategy (from-scratch) and both strategies tied for best (frozen+warm head, fine-tune).

REAL DATA GRAPH



The real, actually-rerun 400-epoch loss curves for the three plotted strategies.

Segment 6 — What This Maps To in PyTorch, and main()

```
def note_on_pytorch():
    print(
        "for p in model.conv1.parameters(): p.requires_grad = False\n"
        "optimizer = torch.optim.SGD(\n"
        "    filter(lambda p: p.requires_grad,\n"
        "        model.parameters()), lr=0.08)"
    )
```

A pure documentation/teaching function, same pattern as every prior day's own `note_on_pytorch()`-style function -- it prints a PyTorch equivalent as a string rather than importing torch. `filter(lambda p: p.requires_grad, ...)` is PyTorch's own version of today's `updates` list: the optimizer only ever steps parameters still marked trainable.

```
def main():
    demo_gradient_check()
    source_net = pretrain_source()
    exp = demo_transfer_experiments(source_net)
    make_comparison_plot(
        exp["losses_scratch"], exp["losses_frozen_warm"],
        exp["losses_finetune_best"],
        exp["results"]["from-scratch"]["test_acc"],
        exp["results"]["frozen+warm head"]["test_acc"],
        exp["results"]["fine-tune (best lr)"]["test_acc"])
    note_on_pytorch()

if __name__ == "__main__":
    main()
```

The script's own entry point, calling everything in the exact order this document has walked it: verify correctness first, pretrain the source network second, run the 5-strategy experiment third (passing the just-pretrained `source_net` straight in as an argument), plot the results fourth, then relate it all back to a production framework. `exp` is the dict `demo_transfer_experiments()` returns -- its string keys ("losses_scratch", "from-scratch", etc.) are exactly the same names this document has used throughout.

Quick Syntax Glossary — New or Reinforced Constructs

`if not freeze_conv: ...` (guarding an entire gradient computation) — New today: an `if` that guards not just a final parameter update but the ENTIRE gradient computation for a group of layers -- when `freeze_conv` is `True`, `maxpool_backward` and `conv2d_backward_mc` are never called at all for the conv layers, not just skipped in their effect.

`getattr(net, name)` — Looks up an attribute (`net.W1`, `net.Wc1`, etc.) using its NAME as a string rather than writing `net.W1` literally -- lets `demo_gradient_check()` loop over a different list of parameter names depending on `freeze_conv` without duplicating the check's logic.

`.copy()` on every warm-started array — Without it, `target_net.Wc1` and `source_net.Wc1` would be the SAME array object in memory -- training the target network would also silently mutate the (supposedly fixed) source network's weights.

`np.array_equal(before, after)` — A literal, non-numerical check that two NumPy arrays are exactly identical -- used today to directly confirm `freeze_conv=True` didn't just skip computing a gradient in theory, but that the actual weight array never changed after real training epochs.

`max(finetune_lrs, key=lambda lr: finetune_results[lr]["test_acc"])` — Picks whichever learning rate in the swept list actually achieved the best real `test_acc`, rather than the script assuming in advance which learning rate would work -- the 'best' fine-tune result reported today is discovered empirically, not chosen ahead of time.

End of Day 48 syntax reference. For the concepts these lines implement, see the main Day 48 study guide PDF.